INSTITUTO FEDERAL
Espírito Santo
Campus Serra

REDES DE COMPUTADORES — PPCOMP

Relatório do Trabalho SDN - Parte 1

Descoberta de Topologias de Rede com Mininet e Ryu

Ana Júlia Caetano Martins — 20261MPCA0143

Professor(a): Cristina Klippel Dominicini

IFES — Campus Serra | 28/10/2026

Sumário

1. Introdução 1
2. Fundamentação teórica 2
2.1. Separação entre Planos e o Protocolo OpenFlow 2
2.2. Mapeamento via Protocolo LLDP 2
2.3. Rastreamento de *Hosts* e *Gratuitous* ARP 3
3. Metodologia 4
3.1. Ambiente experimental 4
3.2. Topologia 4
3.3. Execução e avaliação 5
4. Implementação 6
4.1. Arquitetura e Visão Geral do Sistema 6
4.2. Módulo de Emulação da Topologia 7
4.3. Aplicação de Descoberta de Topologia 7
4.4. Entrada e Saída de Dados 9
5. Resultados e discussão 9
5.1. Topologia triangular 9
5.2. Topologia com múltiplos *loops* 10
5.3. Discussão 12
6. Conclusão 13
Referências 13

1. Introdução

As Redes Definidas por Software (*Software-Defined Networking*, SDN) constituem um paradigma de arquitetura de redes que desacopla o plano de controle, responsável pelas decisões de encaminhamento, do plano de dados, encarregado de sua execução nos equipamentos. Nesse modelo, a lógica da rede é concentrada em um controlador logicamente centralizado, que se comunica com os *switches* por meio de uma interface padronizada, da qual o protocolo OpenFlow é o principal representante. Em decorrência dessa separação, o controlador passa a dispor de uma visão global do estado da rede, o que favorece o desenvolvimento de aplicações de engenharia de tráfego, balanceamento de carga e diagnóstico de falhas (Kurose; Ross, 2021).

A existência dessa visão global pressupõe, entretanto, que o controlador seja capaz de descobrir a topologia sob sua gerência, isto é, identificar os *switches* conectados, os enlaces que os interligam, com as respectivas portas, e o ponto de acoplamento de cada *host*. A descoberta de topologia é, portanto, requisito básico de aplicações que operam sobre o grafo da rede, como o cálculo de rotas, a detecção de *loops* e a recuperação após falhas (Kurose; Ross, 2021). No controlador Ryu, a identificação dos enlaces entre *switches* baseia-se no protocolo LLDP (*Link Layer Discovery Protocol*), ao passo que a localização dos *hosts* é inferida a partir de pacotes por eles, com o exemplo das mensagens ARP (RYU project team, 2016).

A validação experimental de tais aplicações em equipamentos físicos demanda custo e tempo elevados. Como alternativa, o emulador Mininet permite instanciar, em uma única máquina, redes virtuais compostas por *hosts*, *switches* Open vSwitch e enlaces, administráveis por um controlador externo. Esse recurso viabiliza a prototipação de diferentes topologias e a comparação direta entre a rede emulada e a topologia efetivamente percebida pelo controlador (RYU project team, 2016).

Neste contexto, o presente trabalho descreve o desenvolvimento e a validação de uma aplicação de descoberta de topologia para o controlador Ryu (`topo_discover.py`), executada com a opção `--observe-links` . A aplicação mantém o registro dos *switches*, dos enlaces entre eles e dos *hosts* aprendidos por meio de ARP, exibe a topologia a cada novo *host* identificado e informa se o grafo de *switches* é conexo. A validação foi conduzida no Mininet com duas topologias: uma triangular, com três *switches* e um único ciclo, e outra com seis *switches* e múltiplos ciclos, representativa de redes com enlaces redundantes.

O objetivo geral consiste em verificar se a topologia descoberta pelo controlador corresponde à topologia emulada. Os objetivos específicos são: (i) comparar, enlace a enlace e porta a porta, as saídas do Mininet e do Ryu; (ii) analisar a evolução incremental da descoberta de *switches* e *hosts*; e (iii) discutir o comportamento da aplicação na presença de caminhos redundantes.

2. Fundamentação teórica

2.1. Separação entre Planos e o Protocolo OpenFlow

A arquitetura das Redes Definidas por *Software* (SDN) reestrutura o paradigma das redes tradicionais ao desacoplar as funções de decisão (plano de controle) do encaminhamento físico de pacotes (plano de dados). Enquanto no modelo legado os *switches* executam algoritmos distribuídos e tomam decisões de encaminhamento de forma autônoma e descentralizada, no paradigma SDN essa inteligência é centralizada em um elemento lógico denominado controlador. Os dispositivos do plano de dados, como os *switches* virtuais *Open vSwitch* (OVS), tornam-se elementos programáveis voltados estritamente à execução de instruções de encaminhamento enviadas pelo plano de controle (Kurose; Ross, 2021).

No protocolo OpenFlow, o aprendizado autônomo e local dos *switches* legados é substituído pela instalação de regras reativas ou proativas programadas pelo controlador. A comunicação entre os planos se fundamenta na definição de uma tabela de fluxos (*flow table*), uma estrutura de dados composta por um conjunto encadeado de entradas de fluxo (*flow entries*). Cada entrada possui *match fields*, contadores de estatísticas e um conjunto de instruções ou ações a serem aplicadas aos pacotes em velocidade de linha (Kurose; Ross, 2021; Open Networking Foundation, 2012).

Nas tabelas de fluxo, é possível adicionar uma entrada de prioridade mínima (prioridade 0), com *match* vazio, denominada *Table-Miss*. Essa entrada determina o comportamento do *switch* diante de qualquer pacote que não corresponda às regras de maior prioridade. A ação associada ao *Table-Miss* é, tipicamente, o envio de uma mensagem *PacketIn* ao controlador. Trata-se de uma mensagem assíncrona do protocolo OpenFlow, por meio da qual o plano de dados transfere ao plano de controle a custódia total ou parcial do quadro, seja quando ocorre uma exceção no processamento, ou quando há correspondência com a entrada *Table-Miss* (Kurose; Ross, 2021; Open Networking Foundation, 2012; RYU project team, 2016).

2.2. Mapeamento via Protocolo LLDP

No paradigma SDN, a separação entre os planos priva os *switches OpenFlow* de qualquer capacidade autônoma de controle de topologia. Como os dispositivos do plano de dados atuam estritamente como elementos programáveis de encaminhamento, eles não geram nem interpretam quadros de descoberta por iniciativa própria, tornando a reconstrução da infraestrutura física uma responsabilidade exclusiva do controlador (Kurose; Ross, 2021).

Para superar essa limitação e construir uma visão global da topologia *inter-switch*, o *Link Layer Discovery Protocol* (LLDP) é adaptado conceitualmente no ecossistema SDN. Como demonstrado por (Kurose; Ross, 2021), (Open Networking Foundation, 2012) e (RYU project team, 2016), em vez de ser processado localmente em cada nó, o LLDP é utilizado para descobrir as

conexões da rede sob a coordenação do plano de controle. O ciclo conceitual de descoberta de se desdobra nas seguintes etapas:

1. *Packet-Out*: o controlador gera proativamente quadros LLDP contendo em sua estrutura de dados os identificadores do *switch* de origem e da respectiva porta de saída. Esse quadro é encapsulado em uma mensagem de controle do tipo *Packet-Out* e enviado a um *switch* específico, acompanhado da instrução explícita para que seja transmitido pela porta cuja conexão se deseja identificar.
2. Encaminhamento no Plano de Dados: o *switch* receptor executa a instrução do plano de controle, transmitindo o quadro LLDP pelo meio físico de comunicação sem alterar seu conteúdo.
3. *Packet-In*: ao ser recebido pela interface física do *switch* adjacente., o quadro LLDP é interceptado. Como o dispositivo receptor não processa protocolos de controle localmente, o quadro é encapsulado em uma mensagem assíncrona do tipo *Packet-In* e redirecionado imediatamente ao controlador, acompanhado dos metadados referentes à porta física de recepção.
4. Correlação Topológica e Inferência de Enlace: ao receber a mensagem *Packet-In*, o plano de controle realiza a decodificação do pacote. O controlador extrai a tupla de origem (identificador do *switch* emissor e porta de saída) e a correlaciona com a tupla de destino (identificador do *switch* receptor e porta de entrada). A associação direta dessas duas pontas confirma a existência de uma conexão física bidirecional de Camada 2, permitindo ao controlador registrar formalmente o enlace no modelo topológico da rede.

2.3. Rastreamento de *Hosts* e *Gratuitous* ARP

O *Address Resolution Protocol* (ARP) é o protocolo responsável por traduzir endereços lógicos da Camada de Rede (endereços IP) em endereços físicos da Camada de Enlace (endereços MAC) no âmbito de uma rede local (LAN). Como a transmissão de quadros Ethernet exige o endereço MAC de destino, um dispositivo emissor consulta primeiramente sua tabela ARP local. Caso a associação entre o IP e o MAC de destino não conste na tabela, o emissor gera uma requisição de consulta (*ARP Request*) contendo o IP alvo e a transmite encapsulada em um quadro de broadcast para o endereço físico especial `FF-FF-FF-FF-FF-FF`. Todos os nós do mesmo domínio de enlace recebem a mensagem, porém apenas o dispositivo que possui o endereço IP correspondente responde com uma mensagem unicast (*ARP Reply*) informando seu endereço MAC, o que permite ao emissor atualizar sua tabela local e efetivar o envio do tráfego (Kurose; Ross, 2021).

O mecanismo tradicional de descoberta de vizinhança baseado em *flooding* de requisições ARP (*Address Resolution Protocol*) apresenta graves limitações em arquiteturas SDN puras. Na ausência de regras de fluxo pré-instaladas, a transmissão em *broadcast* de um quadro gera um volume massivo de eventos *PacketIn* simultâneos para o controlador. Além disso, em

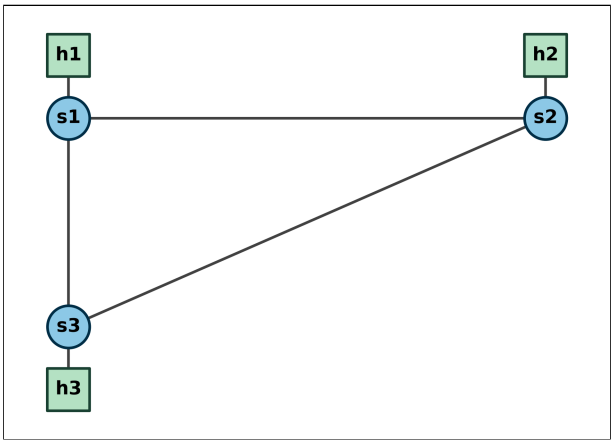


Figura 1: Topologia Triangle.

redes com topologias malhadas ou com enlaces redundantes, a inundação descontrolada induz tempestades de *broadcast* e laços de encaminhamento na Camada 2 antes que o controlador consiga mapear a rede (Kurose; Ross, 2021; RYU project team, 2016).

Para contornar essa limitação e assegurar o rastreamento determinístico dos dispositivos finais sem inundar a infraestrutura, adota-se o *Gratuitous* ARP (GARP) como mecanismo de anúncio de presença. O ARP convencional opera sob a dinâmica de consulta e resposta (*request/reply*). O *Gratuitous* ARP (GARP), por sua vez, consiste em um anúncio não solicitado (*unsolicited* ARP) emitido espontaneamente por um *host*, no qual os campos de IP de origem (SPA) e IP de destino (TPA) no *payload* da mensagem são idênticos ao endereço IP do próprio emissor. Esse processo associa a identidade lógica e física de um nó ao ponto de acesso correspondente na topologia (Cheshire, 2008; Kurose; Ross, 2021; RYU project team, 2016).

3. Metodologia

3.1. Ambiente experimental

Os experimentos foram realizados em uma máquina virtual executada por meio do hipervisor QEMU/KVM, com 4 núcleos de CPU e 4 GB de memória RAM. A máquina virtual utiliza o Ubuntu 20.04.1 LTS, enquanto o sistema hospedeiro utiliza o Ubuntu 26.04.1 LTS.

O ambiente de redes é composto pelo emulador Mininet 2.3.0 e pelo *switch* virtual Open vSwitch 2.13.8. O plano de controle utiliza o controlador Ryu 4.34, implementado em Python 3.8.10, com comunicação via OpenFlow 1.3.0. Além disso, a biblioteca NetworkX 3.1 é utilizada para a representação e manipulação da topologia da rede por meio de grafos.

3.2. Topologia

Para facilitar a depuração e validar o tratamento de grafos arbitrários pela implementação, foram utilizadas duas topologias fixas, representadas pelas Figura 1 e Figura 2. Ambas

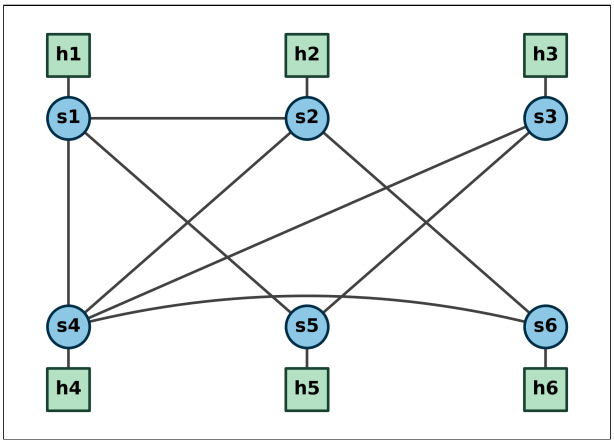


Figura 2: Topologia Someloops.

apresentam múltiplos caminhos e ciclos, permitindo verificar o comportamento do código em diferentes configurações da rede.

As topologias são definidas em arquivos de texto (`.txt`), que especificam os enlaces entre os switches. Além disso, considera-se que cada switch possui exatamente um host conectado.

3.3. Execução e avaliação

Para colocar em execução a aplicação Ryu responsável pela descoberta da topologia (`topo_discover.py`), é utilizado o Comando 1. A *flag* `--observe-links` habilita a descoberta e o monitoramento dos enlaces entre switches, utilizando mensagens LLDP para identificar as conexões entre os dispositivos e suas respectivas portas. Esse mecanismo permite ao controlador manter informações sobre os enlaces entre *switches* e notificar as aplicações sobre mudanças na topologia, como a adição ou remoção de switches, portas e enlaces (SDN Hub, *[S.d.]*).

```bash
ryu-manager --observe-links topo_discover.py
```

Comando 1: Inicialização do controlador Ryu.

Após a inicialização do controlador, é executado o *script* `teste_topologia` , responsável por instanciar, através do Mininet, a topologia definida e iniciar o processo de *Gratuitous ARP* de cada *host*. A flag `-E` é utilizada para a execução preservar as variáveis de ambiente do usuário ao executar o arquivo como *root*.

```bash
sudo -E python3 teste_topologia.py <topologia.txt>
```

Comando 2: Inicialização do Mininet.

A avaliação é realizada por meio da comparação entre a topologia definida no arquivo de entrada, tomada como referência, e a topologia observada em duas fontes: CLI do Mininet e aplicação Ryu. Na CLI do Mininet, a comparação é feita por meio dos comandos `nodes` e `links` , que lista os nós e enlaces efetivamente criados na emulação. Na saída da aplicação Ryu, os *switches* e os enlaces *inter-switch* são obtidos pela descoberta via LLDP e atualizados a cada evento de entrada de *switch* ou enlace. Já os *hosts* são registrados a partir dos pacotes ARP

recebidos em portas de acesso, e a representação completa é impressa a cada ARP reconhecido. Por isso, a verificação dos *hosts* depende do envio de ARP pelos *hosts*, enquanto a verificação dos enlaces depende da conclusão da descoberta LLDP. Para cada topologia, compara-se o conjunto de nós e enlaces descobertos pelo controlador com a entrada (arquivo de texto), registrando-se a proporção de elementos corretamente identificados. Considera-se também o campo de conectividade (`Conexo`) impresso pela aplicação.

4. Implementação

4.1. Arquitetura e Visão Geral do Sistema

A arquitetura comporta três componentes principais. O primeiro é o emulador, responsável por instanciar a rede a partir de um arquivo de topologia. Ele cria *switches* virtuais baseados em Open vSwitch (OVS), um host por *switch* e os enlaces entre eles, e aponta esses *switches* para um controlador remoto. O segundo é o controlador, implementado sobre o framework Ryu, que executa a aplicação de descoberta de topologia. A interação entre esses componentes ocorre em duas fases. Na fase de inicialização, o emulador conecta os *switches* ao controlador e aguarda a descoberta automática dos enlaces entre *switches*, que o controlador realiza com o auxílio do LLDP. Na fase de operação, a aplicação de controle mantém uma representação interna da rede e a atualiza a cada evento relevante.

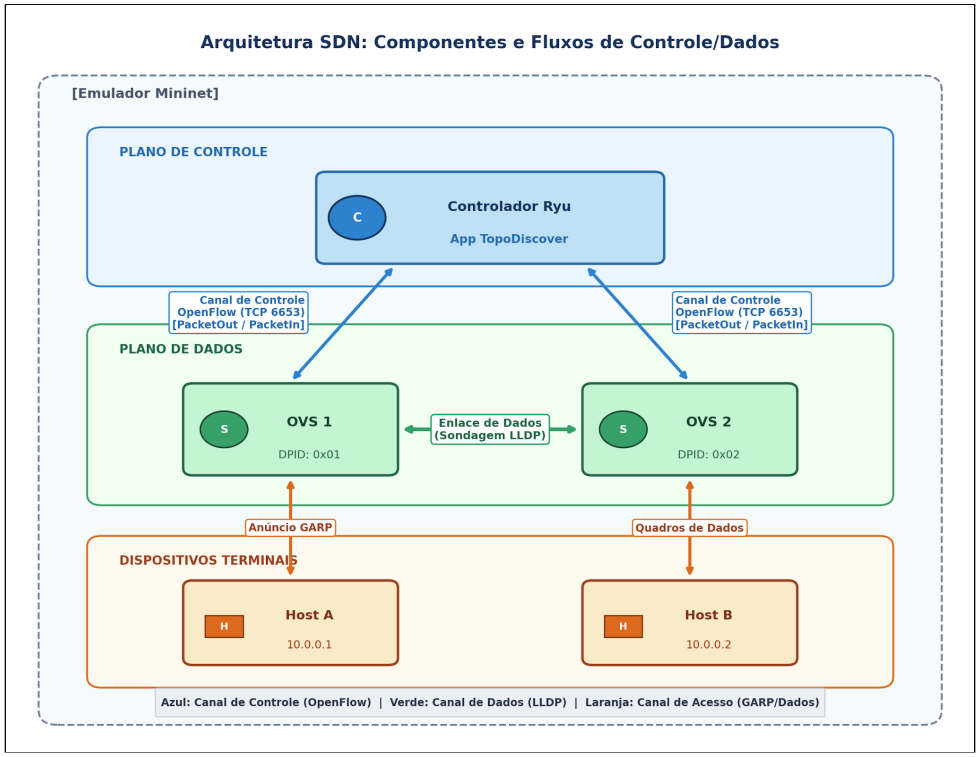


Figura 3: Visão geral dos componentes do protótipo.

A escolha do framework Ryu deve-se à sua implementação em Python, linguagem que dispõe de bibliotecas consolidadas para manipulação de grafos e que se integra diretamente ao

emulador Mininet, também escrito nessa linguagem. Essa homogeneidade reduz a sobrecarga de integração e permite que ambos os componentes compartilhem convenções de nomenclatura e de representação de topologias. A biblioteca NetworkX foi adotada para representar a topologia como grafo não direcionado, o que possibilita o uso direto de algoritmos clássicos de teoria dos grafos, como a verificação de conectividade, sem a necessidade de implementá-los do zero.

4.2. Módulo de Emulação da Topologia

O módulo de emulação transforma uma descrição textual de topologia em uma rede executável. A leitura do arquivo é feita por uma função que constrói um grafo Networkx a partir de uma lista de arestas, ignorando linhas de comentário e tratando os identificadores dos nós como cadeias de caracteres. Em seguida, são removidos os laços, ou seja, arestas que ligam um nó a ele mesmo, uma vez que não representam enlaces físicos entre equipamentos distintos e causariam inconsistências na instanciação dos *switches*.

A inicialização segue uma sequência deliberada. A rede é iniciada e o emulador aguarda cinco segundos antes de prosseguir, intervalo empírico que concede ao controlador tempo para receber as conexões dos *switches* e concluir a descoberta de enlaces por LLDP. Em seguida, cada host envia um *Gratuitous ARP*.

4.3. Aplicação de Descoberta de Topologia

A aplicação de descoberta mantém três estruturas internas: a lista de identificadores de *switches*, a lista de enlaces entre *switches* e um dicionário que associa cada endereço IP a um host descoberto. Cada enlace é registrado como uma quádrupla composta pelo identificador do *switch* de origem, pelo identificador do *switch* de destino e pelas portas utilizadas em cada extremidade. O identificador de um *switch*, denominado *datapath identifier* (DPID), é o identificador lógico atribuído pelo protocolo OpenFlow e constitui a chave que relaciona os eventos recebidos aos dispositivos da rede.

A aplicação `topo_discover.py` foi desenvolvida sobre a API de topologia do Ryu (`ryu.topology.api`). Os *switches* são obtidos por meio da função `get_switch`, que retorna os objetos `Switch` registrados após o estabelecimento da sessão OpenFlow, cada qual identificado pelo respectivo *Datapath ID* (`dpid`). Os enlaces entre *switches* são obtidos por meio da função `get_link`, alimentada pela sondagem via LLDP: o controlador injeta quadros LLDP nas portas dos *switches* por mensagens `PacketOut`; o *switch* vizinho os encaminha ao controlador por `PacketIn`, e o módulo correlaciona a origem (`src.dpid`, `src.port_no`) contida no quadro com o destino (`dst.dpid`, `dst.port_no`) informado pelo `PacketIn`, instanciando um objeto `Link`.

A atualização do estado baseia-se em dois tipos de evento. Eventos de entrada de *switch* e de adição de enlace disparam a sincronização completa da visão de topologia, consultando as informações fornecidas pelo módulo de topologia do próprio Ryu. Essa abordagem evita

manter uma réplica incremental propensa a inconsistências, pois a cada mudança a estrutura é reconstruída. A descoberta de hosts, por sua vez, ocorre quando um *switch* encaminha ao controlador um pacote que não possui regra correspondente. A aplicação instalada em cada *switch* configura uma regra de menor prioridade que direciona tais pacotes ao controlador sem armazenamento em buffer, o que permite inspecioná-los integralmente.

A lógica de identificação de hosts repousa sobre uma distinção estrutural. Um pacote ARP que chega por uma porta utilizada em um enlace entre *switches* provém de outro *switch* e não de um equipamento final, de modo que deve ser ignorado. Ao contrário, um pacote ARP proveniente de uma porta que não conecta *switches* indica a presença de um host ligado diretamente àquele *switch*. O Algoritmo 1 sintetiza esse procedimento.

```
ALGORITMO 1: Registro de hosts a partir de pacotes ARP
Entrada: evento PacketIn ev, com _switch_ d e porta de entrada p
 1  se o pacote não contém cabeçalho ARP então retorne
 2  ip ← ARP.endereço_IP_origem;  mac ← ARP.endereço_MAC_origem
 3  se (d.id, p) pertence ao conjunto PORTAS_ENLACE então retorne
 4  H[ip] ← {mac, dpid: d.id, porta: p}
 5  ATUALIZAR_GRAFO()
 6  IMPRIMIR_TOPOLOGIA()
```

Algoritmo 1: Registro de hosts a partir de pacotes ARP.

O conjunto PORTAS_ENLACE contém todos os pares (*switch*, porta) que participam de algum enlace entre *switches* e é derivado diretamente da lista de enlaces. A reconstrução do grafo, descrita no Algoritmo 2, cria um nó para cada *switch*, um nó para cada host e arestas que representam os enlaces entre *switches* e as conexões entre hosts e *switches*. Cada aresta armazena as portas de ambas as extremidades, o que possibilita reconstruir, a partir do grafo, a informação de porta necessária à análise.

```
ALGORITMO 2: Reconstrução do grafo de topologia
Entrada: conjunto de _switches_ S, lista de enlaces E, dicionário de hosts H
 1  G ← grafo vazio
 2  para cada s em S faça adicionar nó s com tipo 'switch'
 3  para cada (a, b, pa, pb) em E faça adicionar aresta (a, b) com portas {a:
pa, b: pb}
 4  para cada (ip, h) em H faça
 5      adicionar nó ip com tipo 'host' e MAC h.mac
 6      adicionar aresta (ip, h.dpid) com portas {h.dpid: h.porta}
 7  retorne G
```

Algoritmo 2: Reconstrução do grafo de topologia.

Ao término de cada atualização, a aplicação verifica a conectividade do grafo por meio de uma consulta à biblioteca NetworkX. Um resultado verdadeiro indica que existe caminho entre quaisquer dois nós, incluindo *switches* e hosts, enquanto um resultado falso sinaliza descoberta incompleta, seja por enlaces não detectados, seja por hosts ainda não anunciados. Essa verificação funciona como indicador de qualidade da descoberta e é registrada junto com o restante da topologia, de modo a alimentar a análise experimental.

4.4. Entrada e Saída de Dados

O sistema recebe como entrada um arquivo de texto no formato de lista de arestas, em que cada linha descreve um enlace entre dois *switches* por meio de seus identificadores separados por espaço. Linhas iniciadas pelo caractere `#` são tratadas como comentários. O formato foi escolhido por sua simplicidade e por ser o padrão de leitura nativo da biblioteca NetworkX, o que dispensa um analisador específico. A seguir, apresenta-se um exemplo de topologia triangular, com três *switches* interligados:

```
# topologia triangular
s1 s2
s2 s3
s3 s1
```

Listagem 3: Exemplo de arquivo de topologia no formato de lista de arestas.

A saída do sistema é composta por registros emitidos no log do controlador. Ao receber cada anúncio ARP de um host, a aplicação registra o endereço IP, o endereço MAC, o *switch* e a porta de origem, o que permite acompanhar o processo de descoberta em tempo real. Em seguida, emite um bloco estruturado com a topologia completa, contendo a lista de *switches*, os enlaces entre eles com as respectivas portas, a lista de hosts com seus endereços e o *switch* ao qual cada um está conectado, e o valor booleano de conectividade. O formato do bloco é ilustrado a seguir, com valores meramente demonstrativos:

```
================ TOPOLOGIA ================
Switches:
  s1
  s2
Enlaces entre switches:
  s1 (porta 2) <-> s2 (porta 1)
Hosts:
  10.0.0.1 (MAC 00:00:00:00:00:01) -> s1 (porta 1)
Conexo: True
===========================================
```

Listagem 4: Formato do bloco de topologia emitido no log do controlador (valores demonstrativos).

Essa estrutura de registro foi definida para servir diretamente à etapa de avaliação. Como o arquivo de entrada descreve a topologia verdadeira e o log descreve a topologia inferida, a comparação entre ambos permite quantificar a acurácia da descoberta, identificar enlaces ou hosts omitidos e medir o tempo necessário para a convergência, grandeza que pode ser obtida a partir dos instantes de registro dos eventos.

5. Resultados e discussão

5.1. Topologia triangular

A primeira topologia possui três *switches* ligados dois a dois, formando um único ciclo, e um *host* por *switch*. A saída do Mininet confirma a construção esperada:

```text
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
s1-eth2<->s2-eth2 (OK OK)
s1-eth3<->s3-eth2 (OK OK)
s2-eth3<->s3-eth3 (OK OK)
```

Nota-se que a porta 1 de cada *switch* é reservada ao *host* local e que as portas 2 e 3 interligam os *switches*. Do lado do controlador, os *switches* foram registrados à medida que se conectaram:

```text
Switches mapeados atualmente: [1]
Switches mapeados atualmente: [1, 2]
Switches mapeados atualmente: [1, 2, 3]
```

Esse registro incremental mostra que a descoberta de *switches* ocorre de forma assíncrona, conforme cada *datapath* estabelece a sessão OpenFlow com o Ryu. Após o primeiro ARP, a aplicação já exibia todos os enlaces entre *switches*, ainda com um único *host* conhecido:

```text
ARP recebido do IP 10.0.0.1, MAC 52:ca:62:c3:25:89 no Switch 1, porta 1
Enlaces entre switches:
  s1 (porta 2) <-> s2 (porta 2)
  s1 (porta 3) <-> s3 (porta 2)
  s2 (porta 3) <-> s3 (porta 3)
Hosts:
  10.0.0.1 (MAC 52:ca:62:c3:25:89) -> s1 (porta 1)
Conexo: True
```

Comparando com o Mininet, os três enlaces foram descobertos com as portas corretas nas duas extremidades. Isso indica que a descoberta por LLDP, habilitada por `--observe-links`, estava concluída antes da chegada do primeiro ARP, de modo que a parte da topologia referente aos *switches* ficou estável desde o início. Já a lista de *hosts* cresceu conforme os ARPs foram recebidos, até o estado final:

```text
Hosts:
  10.0.0.1 (MAC 52:ca:62:c3:25:89) -> s1 (porta 1)
  10.0.0.2 (MAC da:3a:6d:83:3c:2e) -> s2 (porta 1)
  10.0.0.3 (MAC 96:15:f8:dc:67:2e) -> s3 (porta 1)
```

Cada *host* foi associado ao *switch* e à porta corretos, em concordância com as linhas `h1-eth0<->s1-eth1`, `h2-eth0<->s2-eth1` e `h3-eth0<->s3-eth1` do Mininet. Em todas as impressões, a aplicação reportou `Conexo: True`.

5.2. Topologia com múltiplos *loops*

A segunda topologia tem seis *switches*, oito enlaces entre eles e um *host* por *switch*. Como o grafo possui 8 arestas e 6 vértices, seu número ciclomático é $8 - 6 + 1 = 3$, ou seja, há três ciclos independentes, o que caracteriza uma rede com redundância de caminhos. O Mininet reportou:

```text
mininet> links
s1-eth2<->s2-eth2 (OK OK)
s1-eth3<->s4-eth2 (OK OK)
```

```
s1-eth4<->s5-eth2 (OK OK)
s2-eth3<->s4-eth3 (OK OK)
s2-eth4<->s6-eth2 (OK OK)
s4-eth4<->s3-eth2 (OK OK)
s4-eth5<->s6-eth3 (OK OK)
s5-eth3<->s3-eth3 (OK OK)
```

(além dos seis enlaces *host–switch*, omitidos aqui). A saída final do Ryu, após o último ARP, foi:

```text
Enlaces entre switches:
  s1 (porta 2) <-> s2 (porta 2)
  s1 (porta 3) <-> s4 (porta 2)
  s1 (porta 4) <-> s5 (porta 2)
  s2 (porta 3) <-> s4 (porta 3)
  s2 (porta 4) <-> s6 (porta 2)
  s3 (porta 2) <-> s4 (porta 4)
  s3 (porta 3) <-> s5 (porta 3)
  s4 (porta 5) <-> s6 (porta 3)
Conexo: True
```

A Tabela 1 confronta, enlace a enlace, as duas saídas. Os oito enlaces coincidem, inclusive nos números de porta. A única diferença é de apresentação: o Mininet lista `s4-eth4<->s3-eth2`, enquanto o Ryu imprime o mesmo enlace como `s3 (porta 2) <-> s4 (porta 4)`, pois a aplicação ordena as extremidades pelo identificador do *switch*. Trata-se, portanto, do mesmo enlace.

Enlace (Mininet)	Enlace (Ryu)	Coincide
`s1-eth2<->s2-eth2`	`s1 (porta 2) <-> s2 (porta 2)`	Sim
`s1-eth3<->s4-eth2`	`s1 (porta 3) <-> s4 (porta 2)`	Sim
`s1-eth4<->s5-eth2`	`s1 (porta 4) <-> s5 (porta 2)`	Sim
`s2-eth3<->s4-eth3`	`s2 (porta 3) <-> s4 (porta 3)`	Sim
`s2-eth4<->s6-eth2`	`s2 (porta 4) <-> s6 (porta 2)`	Sim
`s4-eth4<->s3-eth2`	`s3 (porta 2) <-> s4 (porta 4)`	Sim
`s4-eth5<->s6-eth3`	`s4 (porta 5) <-> s6 (porta 3)`	Sim
`s5-eth3<->s3-eth3`	`s3 (porta 3) <-> s5 (porta 3)`	Sim

Tabela 1: Comparação dos enlaces entre *switches* na topologia com *loops*.

A descoberta incremental também se repete nesse cenário. Os seis *switches* foram registrados em sequência, de `[1]` até `[1, 2, 3, 4, 5, 6]`, antes da chegada do primeiro ARP. Assim, já na primeira impressão a aplicação exibia os oito enlaces e apenas o *host* `10.0.0.1`; a cada novo ARP, um *host* era acrescentado:

```text
ARP recebido do IP 10.0.0.4, MAC 6e:99:cc:04:c2:44 no Switch 4, porta 1
Hosts:
  10.0.0.1 (MAC 6a:54:f2:29:ce:27) -> s1 (porta 1)
  10.0.0.2 (MAC 66:be:5d:09:0e:b5) -> s2 (porta 1)
  10.0.0.3 (MAC 82:9b:ef:a2:74:34) -> s3 (porta 1)
  10.0.0.4 (MAC 6e:99:cc:04:c2:44) -> s4 (porta 1)
```

Ao final, os seis *hosts* estavam mapeados às portas 1 dos respectivos *switches*, em acordo com as linhas `h1-eth0<->s1-eth1` a `h6-eth0<->s6-eth1` do Mininet.

5.3. Discussão

Em ambos os experimentos, a topologia descoberta pelo Ryu foi idêntica à emulada no Mininet, tanto nos enlaces entre *switches* quanto na localização dos *hosts*. A Tabela 2 resume as métricas observadas.

Além disso, as saídas confirmam a construção correta das topologias de entrada. Os arquivos de entrada `topo_triangle.txt` e `topo_someloops.txt` descrevem cada enlace como um par de *switches*:

```text
# topo_triangle.txt      # topo_someloops.txt
s1 s2                    s1 s2    s2 s6
s2 s3                    s1 s4    s3 s4
s1 s3                    s1 s5    s3 s5
                         s2 s4    s4 s6
```

O arquivo triangular define 3 enlaces e o arquivo com *loops* define 8, e esses são exatamente os pares de *switches* listados pelo comando `links` do Mininet (3 e 8 enlaces entre *switches*, respectivamente). Não há enlace ausente nem enlace extra: por exemplo, os pares `s3 s4`, `s3 s5` e `s4 s6` do arquivo aparecem no Mininet como `s4-eth4<->s3-eth2`, `s5-eth3<->s3-eth3` e `s4-eth5<->s6-eth3`. O comando `nodes` também lista os 3 e os 6 *switches* esperados, cada um com um *host* associado, e todos os enlaces constam como `(OK OK)`. Como a saída do Ryu coincide com a do Mininet, que por sua vez coincide com os arquivos de entrada, a cadeia de verificação (arquivo de entrada, topologia emulada, topologia descoberta) fica consistente nas duas topologias, o que valida tanto o *script* de construção quanto a aplicação de descoberta.

Métrica	Triangular	Com *loops*
Switches	3	6
Hosts	3	6
Enlaces entre *switches* (Mininet / Ryu)	3 / 3	8 / 8
Ciclos independentes	1	3
Hosts localizados corretamente	3 de 3	6 de 6
Saída `Conexo`	`True`	`True`

Tabela 2: Resumo dos resultados nas duas topologias.

É pertinente destacar que os enlaces entre *switches* já estavam completos na primeira impressão da topologia, o que é coerente com o uso de LLDP pelo módulo `ryu.topology.switches`, que opera de forma proativa. Os *hosts*, ao contrário, só aparecem depois de enviarem tráfego: a aplicação os conhece apenas quando o ARP chega ao controlador. Um *host* silencioso, portanto, permaneceria invisível, o que justifica o envio dos *Gratuitous ARP* no *script* de teste.

Ademais, nas duas topologias, os enlaces redundantes não afetaram a descoberta: cada enlace foi listado uma única vez, sem duplicatas, e a aplicação não registrou ARPs repetidos. Isso é esperado para o ARP inicial, pois cada *host* aparece uma única vez, no *switch* ao qual está diretamente ligado. O valor `Conexo: True` foi impresso em todas as etapas, inclusive nas intermediárias, o que mostra, novamente, que o grafo de *switches* já estava completo antes da descoberta dos *hosts*.

6. Conclusão

Este trabalho validou uma aplicação de descoberta de topologia para o controlador Ryu, comparando o que ela reportou com a rede emulada no Mininet em duas topologias: uma triangular, com um ciclo, e outra com seis *switches* e três ciclos independentes. Em ambos os casos, todos os enlaces entre *switches* foram descobertos com as portas corretas e todos os *hosts* foram associados ao *switch* e à porta em que estão conectados, sem divergências em relação à saída do Mininet. As saídas também confirmam a construção correta das topologias de entrada: os enlaces descritos nos arquivos `topo_triangle.txt` e `topo_someloops.txt` foram todos criados no Mininet, sem enlaces ausentes ou extras, e depois foram corretamente descobertos pelo controlador.

Observou-se que a descoberta ocorre em duas etapas distintas: os *switches* e seus enlaces são identificados logo na conexão ao controlador, por meio do LLDP, enquanto os *hosts* só são conhecidos quando seus pacotes ARP alcançam o controlador. A presença de enlaces redundantes não prejudicou a descoberta, e o indicador de conectividade do grafo de *switches* permaneceu verdadeiro durante todo o processo.

Referências

CHESHIRE, Stuart. **IPv4 Address Conflict Detection**. *[S.l.: S.n.]*. Disponível em: <https://www.rfc-editor.org/rfc/rfc5227.html>.

KUROSE, James F.; ROSS, Keith W. **Computer Networking: A Top-Down Approach**. 8. ed. *[S.l.]:* Pearson, 2021.

OPEN NETWORKING FOUNDATION. **OpenFlow Switch Specification, Version 1.3.0 (Wire Protocol 0x04)**. *[S.l.: S.n.]*. Disponível em: <https://www.opennetworking.org/>.

RYU PROJECT TEAM. **Using OpenFlow 1.3: RYU SDN Framework**. RYU project team, , 26 dez. 2016. Disponível em: <https://ryu.readthedocs.io/>

SDN HUB. **RYU Controller Tutorial**. Disponível em: <https://web.archive.org/web/20200716184816/http://sdnhub.org/tutorials/ryu/>.